Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Developing the System Requirements Specification for Your Own Project

Requirements and System Planning

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 5
Output	System Requirements Specification (SRS), requirements-to-feature traceability matrix, and online requirements review
Maximum Score	80 points

Student / Project Information

Block: BSIT - 3A1 IBPH3	Date: September 21, 2026
System Title: Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification	Instructor: Jean Y. Villasin, MIT
Members: Steven P. Payapaya **Kyle M. Borja**	

1. Activity Alignment and Purpose

Topic Learning Outcome: Analyze the requirements of the group's semester web-system project and produce a scoped requirements specification aligned with the prescribed architecture and selected development methodology.

Course Outcome Alignment: Supports semester-project planning by translating the community problem, user needs, system boundaries, and project constraints into clear and testable requirements that can guide design, implementation, testing, and review.

Syllabus Assessment: System Requirements Specification; requirements-to-feature matrix; online requirements review.

Purpose of the Activity

Activities 2, 3, and 4 established your project architecture, technology stack, and development methodology. In Activity 5, your team will convert the approved community problem and intended user needs into a controlled requirements baseline. You will define functional and non-functional requirements, user roles and permissions, system scope and boundaries, use cases and/or user stories, feature priorities, acceptance criteria, and project constraints. The resulting SRS and traceability matrix will become the reference for the succeeding design, implementation, database, API, testing, and review activities.

Important distinction: A requirement states what the system must do or what quality/constraint it must satisfy. A feature groups related capabilities that users can recognize. Scope defines what the project will and will not deliver. Architecture describes how the solution is organized. Do not replace a requirement with a technology name unless the technology itself is a prescribed project constraint.

2. Intended Learning Objectives

- Analyze the community or organizational problem, target users, stakeholders, and desired system outcomes.
- Write clear, atomic, and testable functional requirements using project-specific actions and data.
- Specify measurable non-functional requirements for security, performance, usability, reliability, compatibility, maintainability, and other relevant quality needs.
- Define user roles, permissions, data-access boundaries, and responsibilities using the principle of least privilege.
- Establish the system scope, out-of-scope items, assumptions, dependencies, and project constraints.
- Refine use cases and/or user stories, feature priorities, business rules, and acceptance criteria based on the selected development methodology.
- Assemble a coherent SRS and requirements-to-feature traceability matrix aligned with Flutter Web, the REST API, PostgreSQL, and the team's selected methodology, then prepare the requirements baseline for online review.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to requirements analysis, refinement, traceability, and review.

Activity Item	Requirement
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled semester-project consultation and online requirements review.
Primary Output	One project-specific System Requirements Specification (SRS) and one Requirements-to-Feature Traceability Matrix.
Supporting Output	Problem/user analysis, functional and non-functional requirements worksheets, role-permission matrix, scope and boundary worksheet, use-case/user-story refinement, feature-priority and acceptance-criteria plan, and review notes.
Recommended Tools	Word/Google Docs, diagrams.net/Draw.io or equivalent for use-case diagrams, spreadsheet/table tool for traceability, Git/GitHub, and the team's current project board or documentation workspace.
Review Format	Online requirements review through the course platform, with instructor consultation or revision requests when requirements are incomplete, ambiguous, out of scope, or inconsistent with the prescribed architecture.

4. Pre-Laboratory Requirements

- Initial list of target users, stakeholders, major features, data to be collected, and any external services or integrations.
- Available requirement evidence such as stakeholder interview notes, observation results, existing forms/processes, instructor feedback, policy/rule references, or project consultations.
- Access to the project GitHub repository or shared documentation workspace where the SRS, diagrams, review notes, and traceability matrix will be maintained.

5. Part A - Community Problem and Target-User Analysis

Begin with the actual problem before writing features. Requirements should be traceable to a real user need, stakeholder rule, project constraint, or system-quality objective. Complete the project-context worksheet and the target-user/stakeholder analysis before defining detailed requirements.

Project Context Item	Your Project Evidence / Statement
Official system title	Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification
Community / organizational problem	Inefficient and manual distribution of humanitarian aid in local government units (LGUs), leading to delayed relief efforts, risk of unfair distribution, and subjective, manual evaluation of household poverty or vulnerability levels.
Current process or problem evidence	LGUs rely on manual paper-based forms and subjective assessment by staff, which can lead to data duplication, delayed aid processing, and inaccuracies in targeting the most vulnerable households.
Target community / organization	Local Government Units (LGUs) and disaster response teams (e.g., in Cabadbaran City).
Primary users / beneficiaries	LGU Staff (field registrars/assessors) and Household Beneficiaries.
Secondary users / stakeholders	LGU Administrators, LGU Disaster Risk Reduction and Management Officers.
Desired system outcome and success indicator	An automated system that collects household assessments, accurately classifies vulnerability levels (High, Medium, Low) using ML, and tracks aid distribution to ensure transparent and equitable relief operations.

Target User and Stakeholder Analysis Worksheet

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
LGU Administrator	Manage user accounts, system configuration, and oversee overall relief distribution and reporting.	Log in, manage system roles, review distribution reports, and inspect vulnerability trends.	Administrative oversight and policy compliance requirements.

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
LGU Staff / Field Assessor	Rapidly register households, collect survey/assessment data, and record distributed relief packs.	Use Flutter Web to register households, fill out assessment forms, check vulnerability results, and record aid distribution.	LGU field operational workflow
Household / Beneficiary	Provide household information and receive fair humanitarian aid based on objective vulnerability criteria.	Receive relief allocations; view/verify eligibility or distribution status via staff assistance.	Community feedback and aid distribution policy.

Write requirements so another team member can implement and test them without guessing. Give each requirement a unique ID. Functional requirements describe system behavior or services. Non-functional requirements describe measurable quality expectations or constraints. Use actual users, data, rules, and project conditions rather than generic statements.

Requirement ID / Type	Requirement Statement	Source / Rationale	Priority	Verification / Acceptance Evidence
FR-001 (Functional)	The system shall authenticate users via username and password and generate a secure token upon successful login.	User Login Workflow	Must	Successful authentication yields HTTP 200 OK with token; invalid returns HTTP 401
FR-002 (Functional)	The system shall allow LGU Staff to register new household profiles with head-of-family and demographic details	Register Household Workflow	Must	Database creation of household record in PostgreSQL (INSERT)
FR-003 (Functional)	The system shall capture household assessment answers and transmit them to the FastAPI REST API	Assessment Workflow	Must	Assessment record saved under PostgreSQL assessments table
FR-004 (Functional)	The system shall integrate with an external ML service to classify household vulnerability as Low, Medium, or High.	ML Integration Feature	Must	Returned classification saved in vulnerability_classifications and displayed on UI
FR-005 (Functional)	The system shall allow LGU Staff to record aid distribution events linked to specific households.	Aid Distribution Feature	Must	New record saved in aid_distributions with timestamp and item details
FR-006 (Non-Functional)	The REST API shall respond to client requests within 2 seconds under standard operations.	Performance / Usability	Must	API latency testing showing response times below 2000 ms.

FR-007 (Non-Functional)	The system shall enforce Role-Based Access Control (RBAC) at the REST API level for protected routes	System Security Boundary	Must	Unauthorized attempts return HTTP 403 Forbidden

Requirement-writing rule

Prefer a testable form such as “The system shall ...”. Keep one requirement per statement. Replace vague words such as fast, secure, easy, user-friendly, reliable, or real-time with measurable conditions whenever the project can define them. A functional requirement should identify the actor or trigger, required behavior, important data or rule, and expected outcome. A non-functional requirement should state the quality attribute, condition, target or threshold, and how it can be verified.

7. Part C - User Roles and Permissions

Define the actual access level of every system role. The role-permission matrix must agree with the use cases/user stories and with the authentication and authorization controls shown in the architecture. Do not give every user full access simply to simplify implementation.

User Role	Purpose / Main Tasks	Allowed Actions	Restricted / Prohibited Actions	Data Access Scope	Authentication / Permission Rule
LGU Administrator	System control, user management, and macro-level report monitoring	Create/manage users, view all households, view reports, trigger distributions	Altering database backend directly or viewing raw private credentials	Full system access across all tables (users, households, reports, etc.)	JWT Auth + Admin Role privileges
LGU Staff	Field operations: household profiling, assessment, and distribution logging	Register households, submit assessments, view vulnerability results, log aid distribution	User management, system configuration changes, deleting logs	Read/Write access to assigned households, assessments, and aid distributions	JWT Auth + Staff Role privileges
Beneficiary	Status verification	View personal vulnerability status / aid receipt confirmation via staff portal	Accessing other households' records or making system edits	Single household record scope	Token/ID restricted query.

Role and access review checks:

- Assign only the permissions required for each role to complete its responsibilities.
- Identify which records a user may view, create, edit, approve, delete, export, or administer.
- Separate sensitive or administrative actions from ordinary user actions and define where authorization must be enforced.
- Ensure each permission is reflected in at least one requirement, use case/user story, or explicit out-of-scope rule.

8. Part D - System Scope, Boundaries, Assumptions, and Constraints

Selected methodology from Activity 4: **Scrum (Adaptive / Agile)**

Prescribed architecture: Flutter Web **->** FastAPI **->** **PostgreSQL**

objective / intended outcome (2-3 sentences):

- Develop a web-based management system to automate household registration, streamline assessment, classify household vulnerability using machine learning, and track humanitarian aid distribution transparently.

Scope and Boundary Worksheet

For each major module or feature area, state what the semester project will deliver, what will remain outside the project boundary, who uses it, what data it needs, and which dependencies or constraints affect implementation.

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
1. Authentication	JWT-based login, role verification	Password self-reset via SMS, biometric login.	Admin, Staff	Credentials, Token	Backend Auth	Completed within Sprint 3.
2. Household Reg.	Demographics collection, address, head of family	Automated GIS mapping of exact GPS coordinates.	LGU Staff	Household info	PostgreSQL DB	Strictly follows IT 108 schema
3. Assessment & ML	Assessment input form, ML-based classification (High/Med/Low)	Training ML models dynamically within the web UI.	LGU Staff	Assessment criteria, Vulnerability score	External ML Microservice	Rest API integration over HTTP
4. Aid Distribution	Recording aid inventory items received by beneficiaries	Electronic payment or digital cash transfers.	LGU Staff	Aid pack details, distribution logs	PostgreSQL DB	Must prevent duplicate distribution
5. Reports & Analytics	Aggregated summary tables, vulnerability status reports	Advanced real-time predictive disaster forecasting.	Admin, Staff	Distribution logs, Vulnerability totals	Backend query logic	Sprint 6 milestone deliverable

Boundary rule
"In scope" means the team commits to analyze, implement, test, and demonstrate the item within the semester. "Out of scope" means it is intentionally excluded from the current version, even if it may be useful in the future. Assumptions are conditions believed to be true; dependencies are external items the system relies on; constraints are limits the team must work within. All four should be explicit to prevent uncontrolled scope growth.

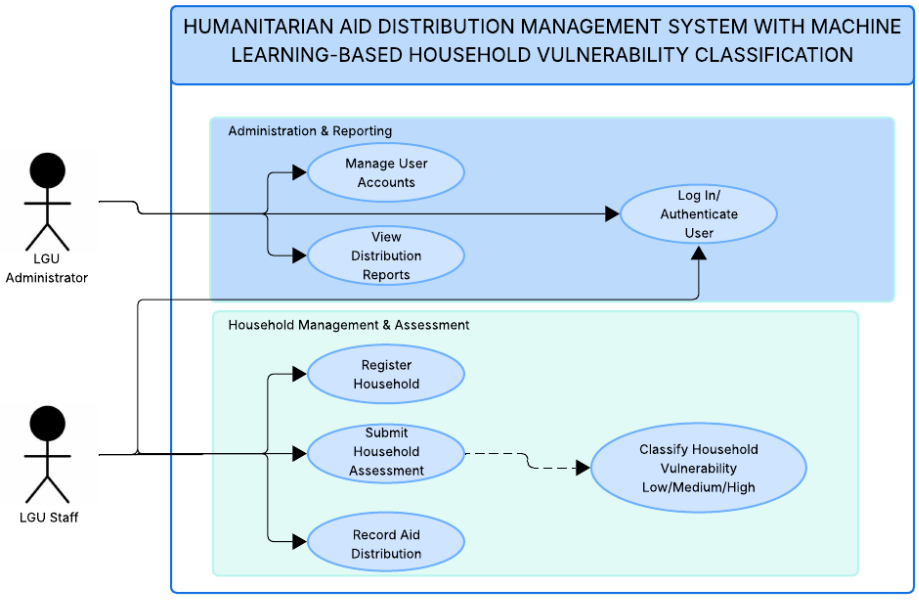
9. Part E - System Use Case Diagram and Use-Case/User-Story Refinement

Model the required behavior from the user's point of view. Every group must first create one complete UML use case diagram for its own approved semester system. The diagram must be based on the actual user roles, functional requirements, scope, and permissions defined in this activity. After the diagram is completed, refine the major use cases and/or user stories in the worksheet below. Teams using a predictive approach may emphasize detailed use-case descriptions, while teams using an adaptive/Agile approach may also maintain user stories and acceptance criteria. The diagram and the written requirements must remain consistent.

Mandatory Use Case Diagram Requirement

Each group must create a UML use case diagram for its own approved system. The diagram is a required Activity 5 output and must be inserted in the SRS or attached as a clearly labeled figure.

- Draw one system boundary rectangle labeled with the official system title.
- Place all major actors or user roles outside the system boundary.
- Show the major user-goal use cases inside the boundary using clear verb-based names (for example: Submit Report, Approve Request, View Map, or Manage Account).
- Connect actors to the use cases they are allowed to perform. Use <<include>> and <<extend>> only when they are logically justified.
- Ensure all Must/High-priority functional features are represented and that the diagram agrees with the role-permission matrix, system scope, and functional requirements.
- Do not place Flutter, the REST API,s PostgreSQL, database tables, screens, or other internal implementation components as actors or use cases. These belong to architecture/design artifacts, not the UML use case diagram.
- Required files: readable PNGof the final use case diagram plus the editable source file (for example .drawio, .fig, or equivalent).



ID	Actor / User	Goal / Need	Trigger / Input	Main Interaction or User Story	Expected Result / Acceptance
UC-01	LGU Staff	Register a new household	Click "New Household" / Form input	Staff fills demographic form and clicks submit	System saves household data into PostgreSQL and returns HTTP 201
UC-02	LGU Staff	Submit assessment and get vulnerability level	Select household / Complete assessment	Staff fills indicators; API posts data to ML service for classification	Classification (High/Medium/Low) displayed and stored
UC-03	LGU Staff	Record distributed aid.	Select household / Input aid items	Staff inputs relief package details and confirms	Aid record appended to database and linked to household
UC-04	LGU Admin	View distribution reports.	Navigate to Reports tab	Admin sets filter dates and clicks "Generate Report"	Summary table of beneficiaries and distributed aid rendered.

Refinement rule
For a use case, identify the actor, goal, precondition when relevant, main flow, alternative/error flow, and expected postcondition. For a user story, use a form such as “As a [role], I want [capability], so that [value]” and attach acceptance criteria. Every Must/High-priority functional requirement should link to at least one use case, user story, or clearly defined system event.

10. Part F - Feature Priorities, Acceptance Criteria, and Project Constraints

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Feature prioritization	Choose and apply one consistent priority scheme such as Must/Should/Could/Won't-for-this-semester or High/Medium/Low.	Must Have (Login, Household Registration, Assessment, Vulnerability ML Classification, Aid Logging); Should Have (Reports, Filters); Could Have (Advanced Dashboard Analytics).
Acceptance criteria	Define observable conditions that must be true before a requirement or feature is considered complete.	A feature is complete only when client-side validation passes, the REST API responds with status code 200/201, PostgreSQL accurately commits transactions, and UI updates state without page crashes.
Business rules	Identify approval rules, calculations, validations, status transitions, eligibility rules, or other domain logic.	A household cannot receive the same aid package twice within a single active distribution event. Assessment records are immutable once saved.
Project constraints	Record deadline, team size, required stack, hardware/device, hosting, data-access, policy, budget, or other fixed limits.	2-member team (Steven & Kyle), semester deadline, mandatory stack (Flutter Web, FastAPI, PostgreSQL).
Data and privacy constraints	State which data are sensitive, who may access them, retention/sharing limits, and minimum protection needed.	Personal demographic data must be protected under standard

		RBAC; no sensitive DB credentials stored in Flutter client.
External-service dependencies	Identify maps/GIS, ML service/model, email/SMS, weather, storage, or other services and what happens when they are unavailable.	Flutter communicates with FastAPI and PostgreSQL; system shows an error if the server is unavailable.
Performance / reliability targets	Define realistic response-time, availability, data-consistency, recovery, or workload expectations when relevant.	API responses should be within 2 seconds; data must remain accurate and consistent Screenshots, API tests, database schema, validation tests.
Validation and error handling	Define required input validation, user feedback, failed-request handling, and recovery behavior.	All required inputs must be validated on both the Flutter client and FastAPI server. Invalid inputs should return clear error messages. Failed API requests must be handled without crashing the app, and users should be informed when an operation fails and given an option to retry.

Recommended minimum requirement evidence in the project repository or shared workspace:

```
docs/requirements/
  system-requirements-specification.docx or .pdf
  requirements-to-feature-matrix.xlsx or .csv
  use-cases-or-user-stories/
  role-permission-matrix/
  scope-and-constraints/
  review-notes/
README.md - current requirements baseline/version and status
```

11. Part G - Construct the System Requirements Specification

Assemble the completed analysis into one coherent SRS. The document must be project-specific and internally consistent. Do not copy generic requirements from another system. Use stable requirement IDs so changes can be traced during implementation and testing.

No.	Required SRS Element	What Must Be Shown
1	Document control and project identity	Official system title, group, version/date, selected methodology, and prescribed Flutter Web-REST API-PostgreSQL architecture.
2	Problem and target users	Community/organizational problem, current situation, intended beneficiaries, primary users, and stakeholders.
3	System objectives	Specific outcomes the system is intended to achieve without describing unsupported future features.
4	Scope and boundaries	In-scope functions/modules, explicit out-of-scope items, assumptions, dependencies, and constraints.
5	User roles and permissions	Role definitions, allowed/restricted actions, data-access scope, and authentication/authorization needs.
6	Functional requirements	Uniquely identified, clear, atomic, feasible, and testable statements of required system behavior.
7	Non-functional requirements	Measurable quality requirements for relevant areas such as security, performance, usability, reliability, compatibility, maintainability, and deployment.
8	Use cases and/or user stories	Actors/roles, user goals, main interactions or stories, alternative/error handling when relevant, and expected results.
9	Features and priorities	Feature list grouped from requirements with a consistent priority scheme and semester-delivery decision.
10	Acceptance criteria and business rules	Observable completion conditions, validations, calculations, approvals, transitions, and other domain rules.
11	Architecture and data alignment	Which requirements are handled by Flutter Web, the REST API/business logic, PostgreSQL, and any approved external services.
12	Traceability and review status	Requirement IDs linked to features/use cases or stories, architecture/data components, acceptance evidence, open issues, and review status.

Minimum SRS Rule

Every Must/High-priority feature must be supported by one or more functional requirements and any relevant non-functional requirements. Each requirement must have a unique ID, a clear source or rationale, a priority, and a verification/acceptance method. Requirements must remain feasible within the selected methodology, semester scope, and prescribed Flutter Web -> REST API -> PostgreSQL architecture.

Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification

System Requirements Specification (SRS)

Document Control and Project Identity

System Title: Humanitarian Aid Distribution Management System with ML-Based Household Vulnerability Classification
Group: BSIT – 3A1 IBPH3
Date: October 07, 2026
Methodology: Agile (Scrum)
Architecture: Flutter Web → REST API → PostgreSQL

Problem and Target Users

Community Problem: delays and inefficiencies in identifying vulnerable households and distributing aid.
Current Situation: manual records and slow assessment lead to unequal aid distribution.
Beneficiaries: vulnerable households, elderly, PWDs, disaster-affected residents.
Primary Users: LGU Administrator, LGU Staff, Households / Beneficiaries.
Stakeholders: LGU, barangay officials

System Objectives

- Centralized and secure platform for household registration and assessment.
- Use machine learning to classify vulnerability levels.
- Enable efficient and transparent aid distribution and tracking.
- Generate real-time reports for decision-making and resource planning.

Scope and Boundaries

In-Scope
- Registration and management
- Vulnerability assessment and classification
- Aid distribution and tracking
- Reports, monitoring, authentication

Out-of-Scope
- Financial transactions / payments
- Physical distribution logistics

Assumptions: internet available; users digitally literate.
Dependencies: Flutter Web, FastAPI, PostgreSQL, external ML service.

User Roles and Permissions

LGU Administrator — manage users/settings; view reports; approve aid distribution. Full access.
LGU Staff — register households; conduct assessments; view assigned data.
Household / Beneficiary — view own profile, check aid status, update household information.
Authentication & Authorization: secure login, JWT tokens, RBAC, session management, audit logs.

Functional Requirements

ID	Requirement
FR-001	Registration and login with valid credentials.
FR-002	LGU staff register and manage household profiles.
FR-003	Collect and store assessment data for each household.
FR-004	Classify household vulnerability using the ML service.
FR-005	Manage aid distribution records and status.
FR-006	Generate reports and analytics for the LGU.
FR-007	Users view their own data and aid status.

Non-Functional Requirements

Security — JWT authentication, RBAC, encrypted data, protection against web attacks.
Performance — < 2s response time; support 100+ concurrent users.
Usability — intuitive responsive UI; accessible on modern browsers.
Reliability — 99% uptime; daily automated backups.
Maintainability — modular code, documentation, Git version control.

Use Cases / User Stories

UC-01 LGU Admin logs in and manages system users.
UC-02 LGU Staff registers households and inputs assessment data.
UC-03 System classifies household vulnerability using ML.
UC-04 LGU Staff creates and tracks aid distribution.
UC-05 Household views profile and aid status.
UC-06 LGU Admin generates reports and analytics.

Example: As a LGU Staff, I want to register a household and submit assessment data so the system can classify its vulnerability level.

Features and Priorities

Feature	Priority	Delivery
User Authentication	High	Phase 1
Household Registration	High	Phase 1
Assessment & ML Classification	High	Phase 2
Aid Distribution	High	Phase 2
Reports & Analytics	Medium	Phase 3
User Management	Medium	Phase 3

Acceptance Criteria and Business Rules

Acceptance Criteria

- Login returns JWT token.
- ML classification returns a valid category.
- Aid distribution is recorded and visible.
- Reports generate accurate data.

Business Rules

- Only LGU Admin can approve aid.
- Each household has one record.
- Vulnerability level: Low / Medium / High.
- Aid must be verified before release.

Architecture and Data Alignment

Component	Handles
Flutter Web (Frontend)	UI, user interaction, authentication (JWT), form validation.
REST API (FastAPI)	Business logic, authentication, ML integration, external API calls.
PostgreSQL (Database)	User data, household data, assessments, aid records, reports.
External ML Service	Household vulnerability classification model.

Traceability and Review Status

Req. ID	Feature / Use Case	Architecture Component	Acceptance Evidence	Open Issues	Review Status
FR-001	Login	Flutter Web + REST API	Successful login with JWT	None	Reviewed
FR-002	Household Registration	Flutter Web + REST API + PostgreSQL	Record saved in DB	None	Reviewed
FR-003	Assessment Input	Flutter Web + REST API + PostgreSQL	Assessment stored	None	Reviewed
FR-004	ML Classification	REST API + ML Service	Correct vulnerability level	Model accuracy ≥ 85%	In Review
FR-005	Aid Distribution	REST API + PostgreSQL	Aid record visible in history	None	Reviewed
FR-006	Reports & Analytics	Flutter Web + REST API + PostgreSQL	Generated report	None	In Review

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the analysis and traceability logic expected in Activity 5. Replace every generic label with the actual community problem, users, requirements, features, roles, constraints, architecture components, and acceptance evidence of your own semester project.

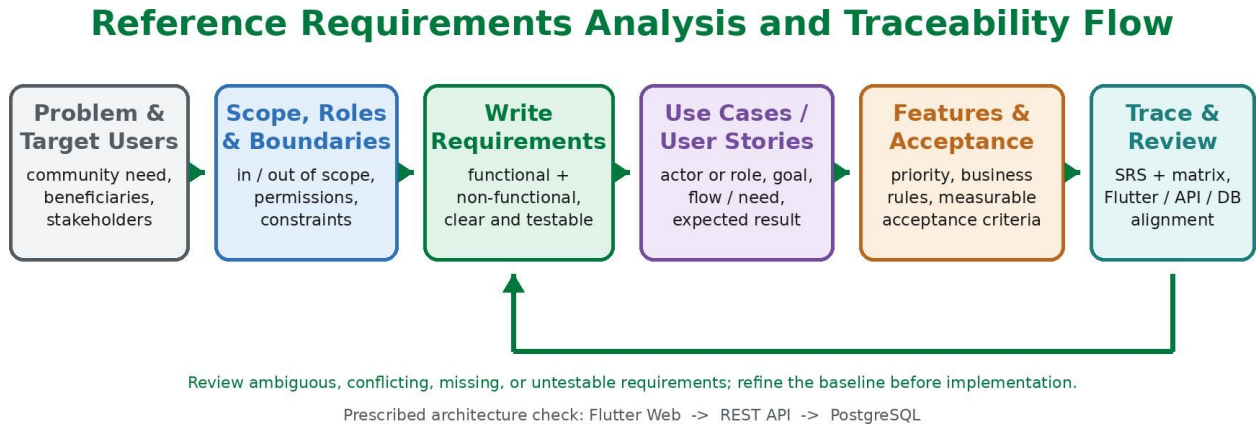


Figure 1. Reference requirements analysis and traceability flow.

Step	Reference Flow
1	Start from the verified community/organizational problem, target users, stakeholders, and requirement evidence.
2	Define system scope, role permissions, boundaries, assumptions, dependencies, and project constraints.
3	Write uniquely identified functional and non-functional requirements that are clear, feasible, and testable.
4	Refine the behavior through use cases and/or user stories that reflect the team's selected development methodology.
5	Group requirements into prioritized features and define measurable acceptance criteria and business rules.
6	Trace requirements to features, Flutter Web/API/PostgreSQL responsibilities, acceptance evidence, and review status; revise gaps before implementation.

13. Part H - Requirements-to-Feature Traceability Matrix

System module / feature set: Authentication, Household Registration, Assessment and ML Vulnerability Classification, Aid Distribution, and Reports & Analytics.

Requirements baseline version / date: _____

Requirement ID	Requirement Summary	Linked Feature & Use Case / Story	Stack Component / Data	Acceptance / Verification & Status
FR-001	The system shall authenticate users using username and password and generate a secure token after successful login.	Authentication / Login	Flutter Web → FastAPI → JWT	HTTP 200 + token for successful login; HTTP 401 for invalid login. Status: Not yet reviewed
FR-002	The system shall allow LGU Staff to register new household profiles with head-of-family and demographic details.	Household Registration / UC-01	Flutter Web → FastAPI → PostgreSQL	Household record successfully created in PostgreSQL. Status: Not yet reviewed
FR-003	The system shall capture household assessment answers and transmit them to the FastAPI REST API.	Assessment / UC-02	Flutter Web → FastAPI → PostgreSQL	Assessment record successfully saved in PostgreSQL. Status: Not yet reviewed
FR-004	The system shall integrate with an external ML service to classify household vulnerability as Low, Medium, or High.	Vulnerability Classification / UC-02	FastAPI → External ML Service → PostgreSQL	Classification is saved and displayed as Low, Medium, or High. Status: Not yet reviewed
FR-005	The system shall allow LGU Staff to record aid distribution events linked to specific households.	Aid Distribution / UC-03	Flutter Web → FastAPI → PostgreSQL	Aid record is saved with timestamp and item details. Status: Not yet reviewed
FR-006	The REST API shall respond to client requests within 2 seconds under standard operations.	System Performance / All major use cases	FastAPI REST API	API latency is below 2000 ms. Status: Not yet reviewed
FR-007	The system shall enforce Role-Based Access Control (RBAC) at the REST API level for protected routes.	Access Control / Login & Authorization	FastAPI → JWT → RBAC	Unauthorized attempts return HTTP 403. Status: Not yet reviewed

14. Part I - Requirements Review and Baseline Approval

Before submission, conduct a structured group review and prepare the SRS for the online requirements review. Record issues that must be revised instead of silently changing the baseline. Confirm all of the following:

- Every requirement is necessary, understandable, uniquely identified, feasible within the semester, and testable or verifiable.
- Functional requirements, non-functional requirements, roles/permissions, scope, use cases/user stories, features, priorities, and acceptance criteria agree with one another.

- No Must/High-priority feature lacks supporting requirements or acceptance evidence, and no requirement is orphaned from a feature, user need, or system-quality objective.
- The requirements are consistent with the selected development methodology and with the prescribed Flutter Web-REST API-PostgreSQL architecture.
- Ambiguous, conflicting, duplicate, missing, or out-of-scope requirements are identified and assigned a revision decision before implementation begins.

Online review status: [ ] Ready for review [ ] Requires revision [ ] Reviewed with comments

Reviewer / consultation source: _____

Date: _____ Baseline version: _____

Most important revision identified during review:

_____

_____

_____

15. Final Quality Checklist

Checklist A	Checklist B
[ ] The community/organizational problem and target users are clearly stated and supported by project evidence.	[ ] Every requirement has a unique ID and uses project-specific, understandable wording.
[ ] Functional requirements describe required behavior rather than vague features or implementation details.	[ ] Non-functional requirements are measurable or verifiable where practical.
[ ] User roles, permissions, restricted actions, and data-access boundaries are consistent across the SRS.	[ ] In-scope and out-of-scope items are explicit; assumptions, dependencies, and constraints are documented.

Checklist A	Checklist B
☑ Use cases and/or user stories are linked to actual roles and required system behavior.	☑ Features are prioritized consistently and acceptance criteria are observable and testable.
☑ Business rules, validation, error handling, and external-service dependencies are documented when relevant.	☑ Requirements are aligned with Flutter Web, the REST API, PostgreSQL, and approved external services.
☑ The requirements-to-feature matrix contains no unexplained orphan requirements or unsupported Must/High features.	☑ The SRS and traceability matrix agree with the selected development methodology and semester plan.
☑ The requirements baseline has been reviewed for ambiguity, conflict, duplication, feasibility, and scope creep.	☑ Every group member can explain assigned requirements, role permissions, acceptance criteria, and traceability decisions during consultation or review.

16. Submission Requirements

- Final System Requirements Specification in editable DOCX and PDF format, using the team's current requirements baseline/version.
- Completed Requirements-to-Feature Traceability Matrix in the SRS or as an accompanying spreadsheet/table file.
- Completed role-permission matrix, scope/boundary worksheet, use-case diagram and/or user-story set, feature-priority list, and acceptance-criteria evidence.
- Updated project GitHub repository or shared workspace containing the requirement artifacts under a folder such as docs/requirements/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA5_SystemName.
- Complete the online requirements review. If the instructor returns comments or identifies requirement defects, submit the revised baseline and preserve a brief record of the changes made.

Academic Integrity and Competency Validation

The SRS must represent the team's own approved project, stakeholder evidence, scope decisions, and technical constraints. AI tools may be used to clarify terminology, refine wording, or check consistency, but generated requirements must not be accepted without verification against the team's actual system. Every member is responsible for understanding the requirements assigned to the group. The instructor may ask any member to explain the source, purpose, priority, permission rule, acceptance criterion, or implementation layer of a requirement without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, ambiguous, inconsistent, untestable, outside the approved scope, or not traceable to the team's actual project. Requirements returned during the online review should be revised before they are used as an implementation baseline.

Criterion	Evidence for Full Credit	Points
Problem, target users, and stakeholder analysis	The SRS clearly identifies the real community/organizational problem, current situation, target users, stakeholders, goals, and evidence used to derive requirements.	8
Functional requirements	Functional requirements are complete for the approved scope, uniquely identified, project-specific, atomic, feasible, clear, prioritized, and testable.	14
Non-functional requirements	Relevant quality requirements are measurable/verifiable and appropriately address security, performance, usability, reliability, compatibility, maintainability, deployment, or other project needs.	10

Criterion	Evidence for Full Credit	Points
User roles, permissions, and scope boundaries	Roles, allowed/restricted actions, data access, in-scope/out-of-scope items, assumptions, dependencies, and constraints are accurate and mutually consistent.	10
Use cases/user stories and feature priorities	Use cases and/or user stories correctly represent user goals and required behavior; features are logically grouped, linked to requirements, and prioritized for the semester.	10
Acceptance criteria, business rules, and constraints	Acceptance conditions are observable/testable; validation, calculations, approvals, errors, external dependencies, and important project constraints are documented when relevant.	8
Requirements-to-feature traceability and architecture alignment	The matrix links requirements to features/use cases or stories, Flutter Web/API/PostgreSQL responsibilities or data, and acceptance evidence without unexplained gaps or contradictions.	12
SRS quality and online requirements review	The document is organized, readable, versioned, internally consistent, submitted correctly, and reviewed for ambiguity, duplication, conflict, feasibility, traceability, and scope before implementation.	8

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which requirement was the most difficult to make clear and testable, and what did your group change?

- The most difficult requirement was the ML-based vulnerability classification because the system must clearly define the input assessment data and expected classification. We clarified that the system will classify households as Low, Medium, or High and save and display the result.

Which requested feature or capability was placed out of scope for this semester, and why?

- Automated GIS mapping using exact GPS coordinates was placed out of scope because the current project focuses on household registration, assessment, vulnerability classification, and aid distribution. It would also add additional complexity to the semester project.

Which user role has the most security-sensitive permissions, and what control should protect those actions?

- The LGU Administrator has the most security-sensitive permissions because the role can manage users, view all household records, and access reports. These actions should be protected using JWT authentication and Admin role-based access control (RBAC)

Which non-functional requirement has the greatest effect on your Flutter Web-REST API-PostgreSQL architecture?

- The performance requirement has a major effect because the REST API must respond within 2 seconds under standard operations. This requires efficient communication between Flutter Web, FastAPI, and PostgreSQL.

What requirement changed or became clearer after the requirements review, and how will your selected methodology manage that change?

- The requirements for validation, error handling, and role-based access became clearer after review. Since the project uses Scrum/Agile**,** changes can be discussed and refined during sprint planning and reviews while keeping the requirements baseline updated.